

Recursion in Java and C/C++

Examples beyond Fibonacci, and where recursion breaks down

These notes continue our recursion discussion with examples that go a step past the Fibonacci case we already covered. Each example is shown in Java and C++ side by side, followed by a look at the situations where recursion causes real problems.

A note before you begin. I assemble these materials myself and revise them often, so an occasional error or typo can slip through. If something does not behave the way I describe when you try it, please tell me.

1. A quick recap

A recursive function needs three things to work correctly.

- At least one base case that returns an answer directly, without another recursive call.
- A recursive case that calls the function again on a smaller or simpler version of the problem.
- Every recursive call must move the problem closer to a base case, or the function never terminates.

2. Example: raising a number to a power

This example multiplies the base by itself, one exponent step at a time. Each call is a smaller version of the same multiplication problem.

Java

```
public static int power(int base, int exponent) {  
    if (exponent == 0) {                // base case  
        return 1;  
    }  
    return base * power(base, exponent - 1);  
}
```

C++

```
int power(int base, int exponent) {  
    if (exponent == 0) {                // base case  
        return 1;  
    }  
    return base * power(base, exponent - 1);  
}
```

This version makes one call per unit of the exponent. A faster version can cut the exponent in half at each step instead of subtracting one, which reaches the base case in far fewer calls. Worth raising as a discussion point rather than showing the code outright.

3. Example: searching a sorted array

This one only works because the array is already sorted. Instead of checking one element at a time, each call throws away half of what is left, which is the same idea raised above as a faster version of power. There are again two base cases: the value is found, or the range to search becomes empty.

Java

```
public static int binarySearch(int[] arr, int target, int low, int high) {
    if (low > high) {
        // base case: not found
        return -1;
    }
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) {
        // base case: found
        return mid;
    } else if (arr[mid] < target) {
        return binarySearch(arr, target, mid + 1, high);
    } else {
        return binarySearch(arr, target, low, mid - 1);
    }
}
```

C++

```
int binarySearch(int arr[], int target, int low, int high) {
    if (low > high) {
        // base case: not found
        return -1;
    }
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) {
        // base case: found
        return mid;
    } else if (arr[mid] < target) {
        return binarySearch(arr, target, mid + 1, high);
    } else {
        return binarySearch(arr, target, low, mid - 1);
    }
}
```

4. Example: counting paths through a grid

The examples so far each made exactly one recursive call. This one makes two, and both of them contribute to the final answer. Picture a grid where you can only move right or down. The number of paths to any square is the number of paths to the square on its left plus the number of paths to the square above it. The base case is the top row or the left column, where there is only one way to get there.

Java

```
public static int countPaths(int row, int col) {
    if (row == 0 || col == 0) {           // base case: edge of the grid
        return 1;
    }
    return countPaths(row - 1, col) + countPaths(row, col - 1);
}
```

C++

```
int countPaths(int row, int col) {
    if (row == 0 || col == 0) {           // base case: edge of the grid
        return 1;
    }
    return countPaths(row - 1, col) + countPaths(row, col - 1);
}
```

5. Example: merging two sorted lists

This makes the same kind of comparison binary search does, but now there are two lists, and each call decides which one to pull from next. It is also a real building block used later in merge sort. There are three things a call can find: both lists are used up, one list is used up, or neither is, in which case the smaller front value is taken.

Java

```
public static void merge(int[] a, int[] b, int[] result, int i, int j, int k) {
    if (i == a.length && j == b.length) { // base case: both used up
        return;
    } else if (i == a.length) {           // base case: only b left
        result[k] = b[j];
        merge(a, b, result, i, j + 1, k + 1);
    } else if (j == b.length) {           // base case: only a left
        result[k] = a[i];
        merge(a, b, result, i + 1, j, k + 1);
    } else if (a[i] <= b[j]) {
        result[k] = a[i];
        merge(a, b, result, i + 1, j, k + 1);
    } else {
        result[k] = b[j];
        merge(a, b, result, i, j + 1, k + 1);
    }
}
```

C++

```
void merge(int a[], int na, int b[], int nb, int result[], int i, int j, int k) {
```

```

    if (i == na && j == nb) {                                // base case: both used up
        return;
    } else if (i == na) {                                    // base case: only b left
        result[k] = b[j];
        merge(a, na, b, nb, result, i, j + 1, k + 1);
    } else if (j == nb) {                                    // base case: only a left
        result[k] = a[i];
        merge(a, na, b, nb, result, i + 1, j, k + 1);
    } else if (a[i] <= b[j]) {
        result[k] = a[i];
        merge(a, na, b, nb, result, i + 1, j, k + 1);
    } else {
        result[k] = b[j];
        merge(a, na, b, nb, result, i, j + 1, k + 1);
    }
}

```

6. Where recursion runs into trouble

Stack growth and stack overflow

Every call keeps a frame on the call stack until it returns, holding its local variables, its parameters, and a return address. Deep recursion, such as merging two very large lists one element per call, can exhaust the stack and crash the program in a way a loop version would not. A missing or incorrect base case makes this worse immediately, since the calls never stop on their own.

```

// Missing progress toward the base case: this never terminates.
public static int brokenPower(int base, int exponent) {
    if (exponent == 0) {
        return 1;
    }
    return base * brokenPower(base, exponent);    // exponent never decreases
}

```

Overhead per call

Local variables, parameters, and the return address are stored for every call, so recursion generally uses more memory than an equivalent loop, and the function-call overhead can make it slower for simple tasks such as searching a short list.

No guaranteed tail-call optimization

Some languages turn certain recursive calls into loops internally so the stack does not grow. Java and standard C++ do not make this guarantee, so a deeply recursive call still consumes stack space in these languages, even when it is written in a tail-recursive style.

Harder to trace

Stepping through a recursive call with a debugger means watching the call stack grow and then unwind, which is a less direct picture than watching a loop variable change on each pass.

Table 1: Recursion versus iteration, at a glance.

Aspect	Recursion	Iteration
Memory per run	Uses one stack frame per call	Uses a fixed amount of memory
Risk of stack overflow	Possible with deep or unbounded recursion	Not applicable
Best fit	Problems that break into smaller versions of themselves	Simple repeated steps
Typical speed	Slower, due to call overhead	Usually faster for simple loops

7. Questions to think about

- Binary search cuts the range in half at every call. About how many calls would it take to search an array of one million elements, compared to checking one element at a time?
- Grid path counting makes two recursive calls that both run, unlike binary search, which only follows one branch. Sketch the call tree for a two by two grid. How many total calls does it make?
- Merge checks whether either list has run out before comparing values. What would go wrong if that check happened after the comparison instead of before it?